Ders 19 Genişletilmiş Notu: Graflara Giriş, Komşuluk Matrisi ve Listesi, Prim ve Kruskal ile En Küçük Kapsayan Ağaç

Ders 19 Genişletilmiş Notu: Graflara Giriş, Komşuluk Matrisi ve Listesi, Prim ve Kruskal ile En Küçük Kapsayan Ağaç

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Graf Nedir, Nerede Kullanılır?
- 2. Graf Terminolojisi
- 3. Kenarın Değeri: Ağırlıklı Graf
- 4. Yönsüz ve Yönlü Graf
- 5. Sembol Tablosu
- 6. Komşuluk Matrisi
- 7. Seyrek Graflar: Tavsiye Sistemi Örneği
- 8. Komşuluk Listesi
- 9. Matris ve Listenin Yer ve Zaman Karşılaştırması
- 10. Yönlü Grafların Gösterimi, Giriş ve Çıkış Derecesi
- 11. En Küçük Kapsayan Ağaç Problemi
- 12. Prim Algoritması
- 13. Kruskal Algoritması
- 14. Komşuluk Listesinin C ile Gerçekleştirilmesi
- 15. Kruskal İçin Etiket Dizisi Tasarımı
- 16. Yönlü Grafta Farklı Ağırlıklar ve Kendine Dönen Kenar

1. Graf Nedir, Nerede Kullanılır?

Hoca, graf (graph) konusunun bugün gerçek dünya uygulamalarında en çok kullanılan veri yapılarından biri olduğunu söyleyerek başladı. Farkında olsak da olmasak da pek çok uygulamada bu veri yapısını kullanıyoruz.

En basit örnek harita uygulamalarıdır. Google Maps veya Yandex gibi bir uygulamaya “Davutpaşa’daki üniversiteden çıktım, Avcılar’a nasıl giderim?” diye sorduğumuzu düşünelim. GPS açıksa başlangıç noktası zaten bellidir. Varış noktasını söylediğimizde uygulama bize birkaç yol gösterir. Yürüyerek ve araçla süreler farklıdır, yol önerileri de farklıdır: “Şuradan gidersen 15 dakika, buradan gidersen parasız yol ama 45 dakika” gibi. Bu planların arkasında şu yapı vardır: Her bağlantı noktası bir düğümdür ve bu düğümler arasında bağlantılar vardır. Yol bulan ve yol tarif eden algoritmaların kullandığı veri yapısı graftır.

Graf, düğümlerden ve bu düğümler arasındaki bağlantılardan oluşur. Bu bağlantılara kenar (edge) denir.

Hoca başka günlük örnekler de verdi:

- **Instagram:** Sizin takip ettiğiniz kişiler ve sizi takip eden kişiler vardır. Bu veri de graf yapısında tutulur.

- **İnternet:** Hocanın gösterdiği şekil, internet üzerindeki bağlantıların görselleştirilmiş hâliydi. İnternet ağı aslında en temel graf yapısıdır.
- **Facebook:** Kişiler ve kişiler arasındaki bağlantılar vardır. Sizin yüz arkadaşınız olabilir. Onların da sizin bağlı olmadığınız elli arkadaşı olabilir.

Bu örneklerin ortak yanı ağdır. Graf genellikle ağ (network) ile gösterilebilen işlemler için kullanılır.

Hoca, temel kavramların Ayrık Matematik (Discrete Mathematics) dersinde görüldüğünü öğrencilerden teyit etti. Bu yüzden terminolojiyi hatırlatma amacıyla hızlı geçeceğini, asıl ağırlığı algoritmaların nasıl geliştirileceğine vereceğini söyledi.

2. Graf Terminolojisi

> **Hocanın vurgusu:** Bu terminolojiyi ezberlemeniz gerekmiyor. Ama gerektiği yerde bilmeniz ve kullanmanız önemli. Düğümü ve kenarı ise tabii ki bilin.

2.1 Düğüm ve Kenar

Grafın iki temel kavramı vardır. Birincisi düğümdür (vertex, çoğulu vertices). Esenler, Davutpaşa, Avcılar gibi yerleri düğümlerle gösterebiliriz. Bir yerden bir yere giden yolu çizdiğimizde uğranılan yerlerin her biri bir düğümdür. İkincisi kenardır. Bir U düğümü ile bir V düğümünün birbirine bağlı olmasını sağlayan yapı kenardır. İki düğüm arasında bağlantı varsa bir kenar vardır.

Soru: Vertices ile node aynı anlamda mı?

Cevap: Evet. Vertex tek bir düğümü, vertices birden fazla düğümü gösterir. Türkçede ikisine de düğüm deriz. Node da aynı şeyin adıdır. Bazıları grafın düğümüne vertex, bazıları node der. İkisi dönüşümlü kullanılır. Node denmesinin sebebi, grafın ağ uygulamalarında kullanılmasıdır. Ağ denince vertex'ten çok node dendiği için bu ad grafa da geçmiştir.

2.2 Derece

Derece (degree), bir düğümün kaç düğüme bağlı olduğunu gösterir. Hocanın slayttaki grafında A düğümünün tek bağlantısı D ileydi, bu yüzden A'nın derecesi 1'dir. B'nin de tek bağlantısı D ileydi, onun derecesi de 1'dir. D ise dört düğüme bağlıydı, derecesi 4'tür. J düğümünün iki bağlantısı vardı, derecesi 2'dir.

> **Ders sonrası not:** Hoca B düğümünden söz ederken bir kez "D'nin derecesi bir" dedi; doğrusu "B'nin derecesi bir"dir. Hemen ardından D'nin derecesini 4 verdi.

2.3 Yol

Farkına varmadan en çok kullandığımız kavram yoldur (path). Esenler'den çıkıp Avcılar'a nasıl gideceğimizi sorduğumuzda aradığımız şey bir yoldur. Yol, bir düğümden başka bir düğüme ulaşmak için izlenen güzergâhtır. Uzunluğu, içerdiği kenar sayısıdır.

Hocanın örneği $A \to B \to C \to D \to E$ yoluydu. A'dan B'ye, B'den C'ye, C'den D'ye, D'den E'ye dört kenar geçilir. Bu yüzden A ile E arasındaki bu yolun uzunluğu 4'tür.

2.4 Döngü

Bir düğümden yola çıkıp aynı düğüme geri dönebiliyorsanız bir kapalı çevrim, yani döngü (cycle) vardır. Örneğin A düğümünden çıkıp dolaştınız ve yine A'ya geldiniz. O zaman grafta bir döngü var demektir.

Hoca bunu A, B, C, D düğümlerinden oluşan küçük bir grafla sordu: Bu grafta döngü var mı? Öğrenciler "yok" dedi. Gerçekten de hangi düğümü başlangıç seçerseniz seçin aynı düğüme geri dönemezsiniz. Sonra hoca bir kenar daha ekledi. Artık B düğümünden çıkıp C üzerinden yeniden B'ye dönülebiliyordu, yani graf döngü içeriyordu.

2.5 Bağlı Bileşen

Hoca bir grafı dünya üzerindeki kıtalara benzetti. Kıtalar birbirinden kopuk ve bağımsızdır. Dünyanın tamamı bir graftır, ama kıtaların her biri ayrı bir parçadır.

Bağlı bileşen (connected component), bir grafta birbirine bağlı olan alt graflardır. Slayttaki grafta çerçevenin içindeki bütün düğümler tek bir grafı gösteriyordu. Bir grafı oluşturan alt grafın tek olması gerekmez. Hepsi birbirine bağlı olsaydı tek bir alt graf olurdu. Burada ise üç alt graf vardı, yani üç bağlı bileşen.

Düğümleri arasında bağlantı olan, tek parça hâlindeki grafa bağlı graf (connected graph) denir. Kendi içinde bağlı parçalardan oluşan, ama bütüne bakıldığında tek parça olmayan grafa bağlı olmayan graf (disconnected graph) denir. Hocanın gösterdiği bağlı olmayan grafta dört bağlı bileşen vardı.

2.6 Ağaç

Yönsüz grafın tanımı sonraki bölümde ayrıntılı gelecek. Şimdilik şu kadarı yeterli: A ile B arasında bir bağlantı gösterdiğimizde A'dan B'ye de, B'den A'ya da gidebiliyorsak graf yönsüzdür (undirected). Yönsüz bir grafta bağlı düğümler arasında döngü oluşmuyorsa bu grafa ağaç (tree) denir. Yani ağaç dediğimiz yapı da aslında bir graftır. Geri dönüş olmayacak, döngü oluşmayacak. Hoca kapsayan ağaç kavramını sonraya bıraktı.

> **Ders sonrası not:** Hoca ağacı "yönsüz ve döngüsüz graf" olarak tanımladı. Tam tanımda ağaç ayrıca bağlıdır. Bağlı olmayan döngüsüz grafa orman (forest) denir.

2.7 İki Parçalı Graf

İki parçalı grafta (bipartite graph) düğümler iki gruba ayrılır. Diyelim ki bir tarafta U grubundaki, diğer tarafta V grubundaki düğümler var. U'daki düğümler kendi içinde birbirine bağlanmaz. V'dekiler de kendi içinde bağlanmaz. Bağlantılar yalnızca iki grup arasındadır.

Hocanın iki örneği şunlardır:

- **Öğrenciler ve dersler:** X öğrencisi matematik ve fizik almış, Y öğrencisi Türkçe ve kimya almış. Öğrencilerin kendi aralarında bağlanmasının bu problemde anlamı yoktur.
- **Seyirciler ve filmler:** Seyircilerin kendi arasında, filmlerin de kendi arasında bağlanmasının anlamı yoktur.

İki grup bilgi varsa ve bunların kendi içinde bağlı olmaması anlamlıysa iki parçalı graf kullanılır. Hoca bunu, graf tasarımı yaparken başvurulabilecek yapılardan biri olarak sundu.

3. Kenarın Değeri: Ağırlıklı Graf

Kenar yalnızca bağlantıyı gösterebilir. Örneğin "Facebook'ta Elif ile Ayşe arkadaş" bilgisi sadece bir bağlantıdır. Ama kenarın bir değeri ya da anlamı da olabilir. Kenarların değeri varsa bu grafa ağırlıklı graf (edge weighted graph) denir.

Hocanın ağırlık örnekleri şunlardır:

- **Mesafe:** Esenler ile Avcılar arası örneğin 10 km.
- **Süre:** O anki trafikte aynı yol 45 dakika.
- **Ulaşım bilgisi:** Hangi otobüse ya da hangi metro hattına bineceğiniz.
- **Maliyet:** Bir öğrencinin önerdiği örnek, gitme maliyeti: 100 lira gibi.
- **Sıfat:** Ağırlığın sayısal olması gerekmez. Elif ile Ayşe arasındaki bağlantı "iş arkadaşı", "okul arkadaşı" veya "kardeş" olabilir. Kenar burada statüyü gösterir.

Sıfat kullanıldığında onu işlerken dönüştürmek gerekebilir. Hoca bunun nasıl yapılacağının örnek uygulamalarda görüleceğini söyledi.

4. Yönsüz ve Yönlü Graf

A, B, C düğümlerini düz çizgilerle bağladığımızda bu bir yönsüz graftır. Burada önemli olan yalnızca A, B ve C'nin bağlı olmasıdır. Yönün önemi yoktur. Facebook'taki arkadaşlık böyledir:

Siz Ayşe ile arkadaşsanız Ayşe de sizinle arkadaştır.

Yön bilgisi önemliyse yönlü graf (directed graph) kullanılır. Esenler'den Avcılar'a giden yolları gösterdiğimizi düşünelim. Tek yönlü ve çift yönlü yollar vardır. A'dan B'ye gidip B'den A'ya dönemiyorsanız bunu göstermeniz gerekir. C ile B arasında iki yönde de gidiş varsa o kenarı iki yönlü gösterirsiniz. Instagram'daki takip de böyledir: Beş bin kişiyi takip ediyor olabilirsiniz, ama onların yalnız beşi sizi takip ediyor olabilir. Takip ettiklerinizin sizi takip etmesi gerekmez.

5. Sembol Tablosu

Grafı bilgisayarda nasıl göstereceğiz? Önce bir sorunu çözmek gerekir. Biz Esenler, Avcılar diye konuşuyoruz, ama bilgi bir dizide ya da matriste saklanacak. Dizide sıfırıncı göz, birinci göz, ikinci göz vardır. "Elif'inci göz" ya da "Ayşe'ninci göz" diye bir şey yoktur.

Bu yüzden iki yönlü bir sembol tablosu (symbol table) gerekir. Esenler'den bahsediliyorsa bu 0 numaralı düğümdür. 0 deniyorsa bu Esenler'dir. Avcılar 1, Zeytinburnu 2 olabilir. Tablonun nasıl kurulacağı ayrı bir konudur ve sizin tasarımınıza bağlıdır. Harf sırasına göre de, haritada başlangıç noktasından ilerleyişe göre de kurulabilir.

Google Maps size 3, 5, 9 gibi numaralar göstermez. Sokak, cadde, semt adı gösterir. Numara görseniz neden bahsettiğini anlayamazsınız, kullanıcının bunu görmesine gerek de yoktur. Uygulama kullanıcıya adları gösterir, arka planda bunları sayıya dönüştürüp işler. Siz de bir tasarım yaparken kullanıcıya "Kelebek Sokak, Ufuk Apartmanı" gibi adları göstermeli, arka tarafta ise sayılarla çalışmalısınız.

6. Komşuluk Matrisi

Grafı gösterebileceğimiz en temel veri yapısı komşuluk matrisidir (adjacency matrix). Yönsüz grafta mantık şudur: 0, 1'e bağlıysa 1 de 0'a bağlıdır. 0, 2'ye bağlıysa 2 de 0'a bağlıdır. Yani bir yansıma vardır ve ikisini de göstermek gerekir.

Satır bir düğümü, sütun öbür düğümü gösterir ve kesiştikleri göz bu iki düğüm arasındaki bağlantıyı tutar. Ağırlıksız grafta iki düğüm bağlıysa göze 1, bağlı değilse 0 yazılır.

Hoca slayttaki grafta 0. düğümü inceledi. 0 düğümü 1, 2, 5 ve 6'ya bağlıydı. Bu yüzden 0. satırın 1, 2, 5, 6 numaralı gözleri 1 olur. Aynı mantıkla 6, 5, 2 ve 1 de 0'a bağlıdır, bu nedenle 0. sütunun aynı gözleri de 1'dir. Slaytta ayrıca 3–5 kenarı için hem `[5][3]` hem de `[3][5]` gözünün işaretlendiği gösterilmişti.

Graf yönsüz ve ağırlıklıysa bağlantı yoksa 0, bağlantı varsa ağırlığın kendisi yazılır. Slayttaki örnekte 0–1 kenarının değeri 2'dir, bu yüzden `[0][1]` ve `[1][0]` gözlerine 2 yazılır. 2–4 kenarının değeri 13'tür, bu yüzden `[2][4]` ve `[4][2]` gözlerine 13 yazılır.

7. Seyrek Graflar: Tavsiye Sistemi Örneği

İkinci gösterim listedir. Önce neden gerektiğini görelim. Hoca bunu Netflix için bir tavsiye sistemi (recommendation system) yazma örneğiyle anlattı. Sayılar derste konuşularak varsayıldı, gerçek değer iddiası değildir.

- **Kullanıcı sayısı:** Hoca önce 1 milyon dedi. Bir öğrenci internetten baktığı 3,5 milyon değerini söyledi. 3,5 milyon alındı.
- **Film sayısı:** 10.000 alındı.
- **Bir kişinin izlediği film sayısı:** Derste söylenen tahminler 20 ile 100 arasındaydı. Hesap için bir satırda 50 dolu göz alındı.

Bu yapıyı matrisle saklamak için, bir kişinin bütün filmlerini tutmak gerektiğinden, 3,5 milyona 10.000'lik matris tanımlamak zorunda kalırsınız. Hesap şöyledir (ara adım eklendi):

$$3{,}5 \times 10^6 \times 10^4 = 3{,}5 \times 10^{10} \text{ göz}$$

Oysa bir satırda yalnızca 50 göz doludur, yani satırın $50/10000 = \%0{,}5$'i kullanılır (ara adım eklendi). Büyük çoğunluğu boş kalan böyle matrise seyrek matris (sparse matrix) denir. Hoca

burada “yarısından fazlası boş” ifadesinin yanıltıcı olduğunu, gerçekten çok çok büyük kısmının boş olması gerektiğini söyledi. Bu yapıyı matriste saklarsanız çok fena yer kaybedersiniz.

7.1 Tavsiye Sistemi Nasıl Çalışır?

Hoca burada konuyu biraz dağıttığını söyleyerek bir soru sordu.

Soru: Tavsiye sistemleri nasıl çalışıyor?

Cevap: Bir öğrenci, sistemin filmleri özelliklerine göre ayırdığını söyledi. Korku filmlerine yüksek oy verdiyseniz izlemediğiniz korku filmlerine de yüksek oy vermişsiniz gibi puanlama yaptığını, en yüksek puanlıları önerdiğini anlattı. Hoca, normalde böyle olması beklendiğini ama sistemin böyle çalışmadığını söyledi. Hocaya göre Netflix’in zamanında öne çıkmasının sebebi bunu yapay zeka ile yapmasıydı.

Hoca sorunu şöyle anlattı. A ve B kişisi ikisi de komedi izliyor olsun. Birisi Cem Yılmaz tarzı filmleri, öbürü Recep İvedik tarzı durum komedilerini seviyor olabilir. İkisine aynı filmi önermemek gerekir. Aksiyon filmlerinde de biri kanlı filmleri sever, öbürü tansiyonu yüksek ama görsel sıkıntısı az olanları tercih eder. Sistem size “kansız aksiyon filmini” nasıl gösterecek?

Çözüm kişilerin benzerliğine bakmaktır. Sistem, kişilerin izledikleri filmlere ve onları beğenip beğenmemelerine bakarak size benzeyen bütün kişileri bulur. Sonra onların izlediği ve beğendiği, ama sizin izlemediğiniz şeyleri size önerir. Üstelik siz yalnız korku izlemezsiniz, komedi de izlersiniz. Çeşidi sizinkine benzeyen insanların beğendikleri önerilir. Hoca bunun benzerlik temelli bir tavsiye sistemi olduğunu ve çeşitli şekilleri bulunduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Öğrencinin anlattığı film özelliklerine dayalı yönteme içerik tabanlı filtreleme (content-based filtering) denir. Hocanın anlattığı kullanıcı benzerliğine dayalı yönteme işbirlikçi filtreleme (collaborative filtering) denir.

7.2 Sosyal Ağ Örneği

Aynı mantık Facebook için de geçerlidir. Hoca kullanıcı sayısını örnek olarak 1 milyar, bir kişinin arkadaş sayısını 100 aldı. 1 milyara 1 milyarlık, yani 10^{18} gözlü (ara adım eklendi) bir matris mi tanımlamalı? Bir kişi en fazla 100 kişiye bağlıyken tabii ki başka bir yol bulmak gerekir. Büyük çoğunluğunun boş olacağını bildiğiniz graflar için matris etkili bir çözüm değildir. Onun yerine liste veri yapısı kullanılır.

8. Komşuluk Listesi

8.1 Ne Zaman Gerekir?

Hoca aradan sonra şu ölçütü netleştirdi. Bağlı olmayan düğüm sayısı çok fazlaysa, örneğin 100 düğümde her düğüm yalnız ikişer üçer düğüme bağlıysa, bu seyrek bir yapıdır. Ama matrisi eleyen tek başına seyreklik değildir, düğüm sayısıyla birlikte düşünmek gerekir:

- **10 düğüm:** $10 \times 10 = 100$ yer gerekir. Yalnız 10’u kullanılsa bile matris kullanın, büyük bir kayıp yoktur.
- **1000 düğüm:** 1000×1000’lik matris de çok dert değildir.
- **100.000 düğüm, her biri en fazla 50 ya da 100 düğüme bağlı:** Artık yer karmaşıklığını (space complexity) düşünüp matrisin etkili olmayacağına karar vermek gerekir.

8.2 Yapısı

Bu durumda komşuluk listesi (adjacency list) kullanılır. Elemanlar bağlı liste (linked list) yapısında saklanır, çünkü her düğümün bağlı olduğu düğüm sayısı farklıdır. Slaytta 0 düğümü dört düğüme bağlıyken 2 düğümü yalnız ikisine bağlıydı.

Hocanın çizdiği beş düğümlü grafın listeleri şöyledir:

```
0 -> 1 -> 2 -> 3
1 -> 0 -> 4
2 -> 0 -> 4
```

```
3 -> 0
4 -> 1 -> 2
```

Bu yapı bir bedel getirir. Bağlı listede 2 düğümüne erişmek için önce 1 düğümünü görmek gerekir. "0, 3'e bağlı mı?" sorusunu yanıtlamak için doğrusal arama (linear search) yaparsınız: 1 mi? Hayır. 2 mi? Hayır. 3 mü? Evet. Yerden kazandınız, ama başka çareniz yoktur.

Soru: Baştaki 0'dan 4'e kadar olan kısım dizi mi?

Cevap: Evet. O bir bağlı liste dizisidir. Dizinin her elemanı, bağlı olunan düğüm sayısı farklı olduğu için, ayrı bir bağlı listeyi gösterir.

8.3 Ağırlıklı Grafta Liste

Kenarların ağırlığı varsa liste elemanı üç parçalı bir yapı (struct) olmalıdır: bağlı olunan düğümün numarası, bağlantının değeri ve bir sonraki elemanın adresi. Hocanın aynı graf üzerinde verdiği örnekte 0–1 kenarının değeri 5, 0–2 kenarının değeri 2, 0–3 kenarının değeri 7'ydi.

> **Hocanın vurgusu:** Kodlamaya girmeden önce mantığını anlamanız önemli.

9. Matris ve Listenin Yer ve Zaman Karşılaştırması

9.1 Yer

V düğüm varsa matriste her düğümün diğerine göre bağlantısı gösterileceği için $V \times V$, yani V^2 yer gerekir. Listede ise yalnızca var olan bağlantılar tutulur ve gereken yer $V + E$ olur.

Hoca bunu 3 düğümlü, 2 kenarlı küçük bir grafla gösterdi: 0, 1'e ve 2'ye bağlı; 1, 0'a bağlı; 2, 0'a bağlı. Kenar sayısı 2'dir, ama graf yönsüz olduğu için 0–1'i gösterdiğiniz gibi 1–0'ı da göstermek gerekir. Bağlantılar 0–1, 1–0, 0–2, 2–0 olmak üzere dört tanedir. Hoca bu nedenle burada her yönün ayrı sayılabileceğini söyledi.

> **Ders sonrası not:** Hocanın $V+E$ ifadesinde yönsüz graf için E, her yönü ayrı sayar. Kenar sayısı E ise listelerde $2E$ kayıt bulunur ve toplam $V + 2E$ yer gerekir, yani $O(V + E)$. Bu örnekte $3 + 4 = 7$ yer, matriste ise $3^2 = 9$ yer gerekir.

9.2 Zaman

Hoca üç soruyu karşılaştırdı.

Soru	Komşuluk matrisi	Komşuluk listesi
v ile w bağlı mı?	$O(1)$	v'nin listesi boyunca doğrusal arama
v'nin bağlı olduğu herhangi bir düğüm var mı?	$O(V)$	$O(1)$
v'nin bağlı olduğu bütün düğümler	Satır taranır, $O(V)$	Yalnız komşular gezilir

v ile w bağlı mı? Matriste doğrudan o göze gidip bakarsınız. Değer 1 ise bağlıdır, değilse bağlı değildir. Listede ise sırayla bakmak zorundasınız. "0, 2'ye bağlı mı?" sorusunda 0'ın listesinde önce 1'e, sonra 2'ye bakarsınız.

> **Ders sonrası not:** Hoca en kötü durumu "kenar sayınız kadar" diye söyledi. Doğrusu, toplam kenar sayısı değil, v'nin listesindeki eleman sayısı, yani v'nin derecesidir. Derece en fazla $V - 1$ olabilir.

v'den herhangi bir düğüme bağlantı var mı? Matriste en kötü durumda bütün satırı tarayıp sıfırdan farklı bir göz aramanız gerekir, bu yüzden maliyet V'dir. Listede liste başına bakmak yeterlidir. Baş `NULL` ise düğüm kimseye bağlı değildir. `NULL` değilse, kime bağlı olduğu önemli olmadan, birilerine bağlıdır. Maliyet $O(1)$'dir.

Bağlı olduğu bütün düğümleri ver. Listede zaten yalnız bağlı olunan düğümler tutulduğu için doğrudan okunurlar.

Ders sonrası not: Hoca bu soru için matriste "bütün matrisi dolaşmanız gerekir" dedi. Tek bir düğüm için yalnız o düğümün satırını taramak yeterlidir.

Hocanın vurgusu: Liste mi matris mi kullanacağınıza, tasarlayacağınız algoritmanın ihtiyaçlarına bakarak karar vermelisiniz.

Doğrudan erişim çoğu zaman matrisi elverişli kılar. "3, 5'e bağlı mı?" sorusunda 3. satırın 5. gözüne bakarsınız. Bu ihtiyacın çok olduğu bir tasarımda matris avantajlıdır. Ama binlerce kişi var da siz yalnız 3–5 kişiye bağlıysanız liste avantajlıdır.

10. Yönlü Grafların Gösterimi, Giriş ve Çıkış Derecesi

Yönlü graflarda da kullanılan veri yapıları matris ve listedir. Tek fark gösterim şeklindedir.

Yön, örneğin 0'dan 1'e gidilip 1'den 0'a dönülemediği anlamına gelir. Hocanın örneği: Kelebek Caddesi'nde trafik tek yönlü aksın. Kelebek'ten Martı Caddesi'ne gidersiniz, ama Martı'dan Kelebek'e dönmek için başka bir yol kullanmanız gerekir. Google Maps'in yapısı tek yönlü ve çift yönlü yollar içerdiği için yönlü graftır.

Web sayfalarındaki bağlantılar (hyperlink) da yönlü graftır. Bir sitede başlangıç noktasından çıkıp tıklaya tıklaya ilerlersiniz. Hocanın slaytındaki örnekte 8 numaralı sayfadan 48'e, 48'den 21'e gidiliyordu. 21'den 48'e bir bağlantı yoktu, geri ancak tarayıcının geri tuşuyla dönülebilirdi.

10.1 Giriş ve Çıkış Derecesi

Yönsüz grafta yalnızca dereceden söz edilirdi. Yönlü grafta iki ayrı derece vardır: giriş derecesi (in degree) düğüme gelen kenar sayısıdır, çıkış derecesi (out degree) düğümden giden kenar sayısıdır.

Slayttaki 6 numaralı düğüme yalnız 7'den gelinir, bu yüzden giriş derecesi 1'dir. 6'dan 0'a, 4'e ve 9'a gidilir, bu yüzden çıkış derecesi 3'tür.

10.2 Matris ve Liste

Matriste artık simetri yoktur. Satırlar başlangıcı, sütunlar gidilen düğümü gösterir. `A[v][w]` gözü v düğümünden w düğümüne olan bağlantıyı sorgular. Slayttaki grafta 5'ten 3'e kenar vardı, ama 3'ten 5'e yoktu. Bu yüzden `[5][3]` gözü 1, `[3][5]` gözü 0'dır.

Listede yalnızca o düğümden gidilebilen düğümler tutulur. Slaytta 0'ın listesinde 5, 2, 1 ve 6 vardı. 5'in listesinde ise yalnız 4 ve 3 vardı, 0 yoktu. Graf yönsüz olsaydı 0, 5'e bağlıyken 5 de 0'a bağlı olurdu ve her kenar için iki kayıt gerekirdi.

10.3 Havaalanı Örneği

Hoca sembol tablosu konusuna bir slaytla geri döndü. Slaytta havaalanları için kullanılan bir graf yapısı vardı. 0 denince bir havaalanı kodu, 1 denince başka bir kod (MCO) anlaşılıyordu. Uygulamada kullanıcıya kodlar gösterilir, arka tarafta dizi kullanılır.

11. En Küçük Kapsayan Ağaç Problemi

Grafların çok kullanıldığı alanlardan biri en kısa yolu bulma gibi bağlantıları değerlendiren uygulamalardır. Hoca bunların arasından şu problemi seçti: Bir ağ üzerinde cihazlarımız var ve hepsinin birbirine bağlı olmasını istiyoruz. Bağlılıktan kastımız, çift yönlü bağın önemli olması değil, ağda kopukluk kalmamasıdır. Bağlantıyı örneğin fiber optikle kuracağız. Masraf az olsun diye en az mesafeyi sağlayacak bağlantıyı seçmek istiyoruz.

İstediğimiz şey iki maddedir: Bütün düğümler birbirine bağlı olacak ve bu bağlantı en az masrafla sağlanacak. Bir düğümü bir kez bağladıysanız o işin içindedir, başka bir yerden tekrar bağlamanız gerekmez.

Bu yapıya en küçük kapsayan ağaç (minimum spanning tree, MST) denir. Kavramları hoca sırayla kurdu:

- **Ağaç:** Yönsüz ve döngü oluşturmayan (acyclic) graf. Bir kenar ekleyip döngü oluşturduğunuz anda artık ağaç değildir.
- **Kapsayan ağaç (spanning tree):** Graftaki bütün düğümleri içeren ağaç.
- **En küçük kapsayan ağaç:** Seçilen kenarların toplam ağırlığı en küçük olan kapsayan ağaç.

Böylece üç şart vardır: bütün düğümler yer alacak, döngü oluşmayacak, seçilen kenarların toplamı en küçük olacak. Bunu yapmak için iki temel algoritma vardır: Prim ve Kruskal.

Hoca iki algoritmayı aynı graf üzerinde gösterdi. Graf 7 düğüm ve 11 kenarlıdır:

```
A-B 7    A-D 5    B-C 8    B-D 9    B-E 7    C-E 5
D-E 15   D-F 6    E-F 8    E-G 9    F-G 11
```

12. Prim Algoritması

Prim açgözlü (greedy) bir algoritmadır. Açgözlü algoritmanın görüşü şudur: Yapının tamamına, global yapıya bakmadan, her adımda yerel olarak en iyi seçeneği seçerek ilerlersem, iş bittiğinde globalde de en iyi sonuca ulaşırım.

Prim buna göre şöyle der: Bir düğümden başlarım. Ağacı her seferinde bir düğüm ekleyerek büyütürüm. Eklediğim düğüm her zaman olabilecek en küçük ağırlıklı bağlantıyı sağlamalıdır ve döngü oluşturmamalıdır.

12.1 Adım Adım Çözüm

Başlangıç düğümü A'dır. Hoca her adımda ağaca giren düğümün bağlantılarını da yazarak ilerledi.

Adım	Ağaçtaki düğümler	Ağaca bağlanabilecek kenarlar	Seçilen	Ağırlık	Ara toplam
1	A	A-B 7, A-D 5	A-D	5	5
2	A, D	A-B 7, B-D 9, D-E 15, D-F 6	D-F	6	11
3	A, D, F	A-B 7, B-D 9, D-E 15, E-F 8, F-G 11	A-B	7	18
4	A, B, D, F	B-C 8, B-E 7, D-E 15, E-F 8, F-G 11	B-E	7	25
5	A, B, D, E, F	B-C 8, C-E 5, E-G 9, F-G 11	E-C	5	30
6	A, B, C, D, E, F	E-G 9, F-G 11	E-G	9	39

Tablodaki aday sütunu, iki ucundan yalnız biri ağaçta olan kenarları listeler (ara adım eklendi). Hoca ise tahtada ağaca giren düğümlerin bütün bağlantılarını yazıp döngü oluşturanları tek tek eliyordu. Bu eleme son adımda belirgindi:

- En küçük kalan değer 8'dir. E-F 8 alınmalı mı? Hayır, E de F de zaten ağaca bağlı olduğundan döngü oluşur.
- İkinci 8 B-C'dir. B ve C de zaten alınmış düğümlerdir. Graf yönsüz olduğu için C-B de aynı kenardır, o da alınmaz.
- 9'lara gelindi. B-D alınamaz, iki ucu da ağaçtadır.
- Diğer 9 E-G'dir. G henüz bağlı olmadığı için alınır.

Bağlantı yaparken dikkat edilen kural budur: Bağlıysa bir daha bağlamamak gerekir. C, E-C seçilirken kimseye bağlı değildi, bu yüzden alınabildi.

Toplam $5 + 6 + 7 + 7 + 5 + 9 = 39$ çıkar. Hoca da 11, 18, 25, 30, 39 diye toplamıştı.

12.2 Nerede Biter?

Bütün düğümler bağlandığı anda en küçük kapsayan ağaç bitmiştir. Bunun sayısal karşılığı şudur: n düğüm varken $n-1$ kenar olduysa iş bitmiştir. Burada 7 düğüm ve 6 kenar vardır.

Soru: Hangi kenarın alınacağını daha sonra programsal olarak görecek miyiz? Şekilde görmek kolaydı.

Cevap: Hoca, Kruskal daha çok kullanıldığı için algoritmik tasarım olarak Kruskal'ı anlatacağını söyledi. Prim üzerinde öğrencilerin kendilerinin oynayarak çalışabileceğini belirtti.

13. Kruskal Algoritması

13.1 Hocanın Prim ile Karşılaştırması

Hoca Kruskal'a geçerken Prim'in yerel en iyiyi seçtiğini hatırlattı ve şöyle dedi: Bu olası bir çözümdür, ama başlangıç noktasına bağlı olarak her zaman en iyi ağacı vermeyebilir. Örneğin bu grafta 8'lik kısa bağlantılar alınamadı, başka bir noktadan başlansaydı belki alınabilirdi. Açgözlü olması çok kolay çözüm üretmeyi sağlar, ama globalde en iyi sonucun garantisi yoktur. Kruskal ise global olarak ideal çözümü bulmaya yöneliktir.

> **Ders sonrası not:** Hoca, Prim'in başlangıç düğümüne bağlı olarak en küçük kapsayan ağacı vermeyebileceğini söyledi; doğrusu, bağlı ve ağırlıklı bir grafta Prim hangi düğümden başlarsa başlasın bir en küçük kapsayan ağaç bulur. Kruskal da açgözlüdür ve o da her zaman bir en küçük kapsayan ağaç bulur. Eşit ağırlıklı kenarlar varsa iki algoritma farklı ağaçlar bulabilir, ama toplam ağırlık aynıdır. "Açgözlü yöntem her problemde en iyi sonucu garanti etmez" sözü genel olarak doğrudur, ancak bu problem için garanti vardır.
>
> **Ders sonrası not:** Hocanın 8'lik kenarlarla ilgili tahmini bu graf için de geçerli değildir. Bu grafın 141 kapsayan ağacı programla tek tek sayıldı. En küçük toplam 39'dur ve bu toplamı veren tek bir ağaç vardır: A-B, A-D, B-E, C-E, D-F, E-G. Prim yedi düğümün her birinden başlatıldığında hep bu ağacı verdi. Yani 8'lik kenarlar hiçbir başlangıçta alınmaz.
>
> **Ek bilgi (derste anlatılmadı):** İki algoritmanın da doğruluğu kesit özelliğine (cut property) dayanır. Düğümleri iki gruba ayıran herhangi bir kesitte, iki grup arasındaki kenarların en küçüğünü alan bir en küçük kapsayan ağaç vardır. Prim'de kesit, ağaçtaki düğümlerle diğerleri arasındadır. Kruskal'da seçilen kenar, kendi alt grafını grafın geri kalanından ayıran kesitin en küçük kenarıdır, çünkü daha küçük kenarlara daha önce bakılmıştır.

13.2 Kenarların Sıralanması

Kruskal'da önce kenarlar ağırlıklarına göre küçükten büyüğe sıralanır. Hedef yine olabildiğince küçük bağlantıları almak, ama döngü oluşturmamaktır. n düğüm varsa $n-1$ bağlantı alındığında iş tamamdır.

Sıralı liste şöyledir:

```
A-D 5   C-E 5   D-F 6   A-B 7   B-E 7   B-C 8
E-F 8   B-D 9   E-G 9   F-G 11  D-E 15
```

13.3 Etiket Dizisiyle İzleme

Hangi kenarın alınıp alınmadığını nasıl izleyeceğiz? Bunun için bir etiket dizisi tutulur. A, B, C, D, E, F, G düğümleri için dizinin indisleri 0, 1, 2, 3, 4, 5, 6'dır: 0. indis A, 1. indis B düğümünü gösterir. Başlangıçta hiçbir düğüm alınmadığı için bütün etiketler sıfırlanır.

```
Kenar      Ağ.   A B C D E F G   Alınan  Toplam  Karar
Başlangıç        0 0 0 0 0 0 0   0       0
A-D        5     1 0 0 1 0 0 0   1       5       Alındı, etiket 1
C-E        5     1 0 2 1 2 0 0   2       10      Alındı, etiket 2
D-F        6     1 0 2 1 2 1 0   3       16      Alındı
```

```
A-B       7     1 1 2 1 2 1 0   4       23      Alındı
B-E       7     1 1 1 1 1 1 0   5       30      Alındı, 2'ler 1 oldu
B-C       8     1 1 1 1 1 1 0   5       30      Alınmadı, döngü
E-F       8     1 1 1 1 1 1 0   5       30      Alınmadı, döngü
B-D       9     1 1 1 1 1 1 0   5       30      Alınmadı, döngü
E-G       9     1 1 1 1 1 1 1   6       39      Alındı
```

Hocanın anlatımıyla adımlar şöyledir:

- **A-D:** İkisi de sıfırdır, yani daha önce alınmamıştır. İlk kez etiketleme yapıldığı için etiket 1 verilir.
- **C-E:** İkisi de alınmamıştır, onlar da alınabilir. Ama etiket 1 kullanıldı ve bu yeni bir bağlantıdır, bu yüzden 2 ile etiketlenir. Artık iki alt graf vardır: A-D ve C-E. Bunlar birbirine bağlı değildir.
- **D-F:** D, 1 numaralı alt grafa bağlıdır, F kimseye bağlı değildir. F de 1 numaralı alt grafa eklenir.
- **A-B:** Aynı şekilde B de 1 ile etiketlenir. Bu noktada A, B, D, F bir alt graf, C, E ikinci alt graftır. 7 düğüm için 6 kenar gerekir, şu ana kadar 4 kenar alınmıştır.
- **B-E:** B'nin etiketi 1, E'nin etiketi 2'dir. Farklı etiket iki ayrı alt graf demektir. İş bittiğinde tek bir graf istendiği için kenar alınır. Dizide bunu göstermek için bütün 2'ler 1 yapılır ya da bütün 1'ler 2 yapılır. Hoca E ve C'yi 1 yaptı.
- **B-C:** İkisi de 1 ile etiketlidir, yani aynı alt graftadır. Hoca çizim üzerinde B-C eklenince döngü oluştuğunu gösterdi. Alınmaz.
- **E-F ve B-D:** Aynı mantıkla ikisi de döngü oluşturur. Alınmaz.
- **E-G:** G'nin etiketi 0'dır, yani henüz hiçbir alt grafa girmemiştir. G, E'nin alt grafına dahil edilir ve 6. kenar alınır.

7 düğüm ve 6 kenarla iş biter. F-G ve D-E kenarlarına bakılmaz. Sonuç, bütün düğümlerin bağlı olduğu ve döngü içermeyen bir kapsayan ağaçtır. Toplam $5 + 5 + 6 + 7 + 7 + 9 = 39$'dur (hocanın sayımı: 5, 10, 16, 23, 30, 39). Prim'de bulunan değerle aynıdır.

14. Komşuluk Listesinin C ile Gerçekleştirilmesi

Hoca, Kruskal'ın algoritmik tasarımından önce komşuluk listesinin nasıl gerçekleştirileceğini slayt üzerinden anlattı. En basit hâlden başladı: Ağırlık yok, her düğüm için yalnız numarası ve gösterdiği yer saklanıyor.

14.1 Yapılar

Liste elemanı (`struct node`) iki parçadır. `vertex` düğümün numarasını, `next` bir sonraki elemanın adresini tutar. 0'ın listesinde sırayla 1, 2, 3 elemanları ucu ucuna eklenir.

Graf yapısı (`struct Graph`) ise düğüm sayısını ve `adjLists` adlı listeyi tutar. Düğüm sayısını bilmek önemlidir, çünkü o uzunlukta bir bağlı liste dizisi tanımlanacaktır. `adjLists` çift işaretçidir (double pointer), çünkü listenin her elemanı bir liste gösterir. 9 düğüm varsa dizinin uzunluğu 9'dur. Hoca bunu, programlama derslerinde görülen matris tanımlama mantığına benzetti, ama burada her göz kendi içinde bir bağlı listedir.

14.2 `createGraph`

Önce graf için yer ayrılır. Düğüm sayısı parametre olarak gelen `vertices` değeridir. Sonra komşuluk listesi için düğüm sayısı kadar, `struct node*` türünde yer tutulur. Başlangıçta hiçbir düğüme başka bir düğüm bağlı olmadığı için bütün gözler `NULL` yapılır. Yeni kenarlar geldikçe bu değerler değişir.

14.3 `createNode`

Yeni düğüm iki parçadan oluşur. `vertex` alanına numara yazılır: "Sen 3 numaralı düğümsün." `next` alanı ise `NULL` olur, çünkü düğüm şu anda boşlukta durur, hocanın deyimiyle "havada uçar", kimseyi göstermez.

14.4 `addEdge` ve Başa Ekleme

Örneğin 0 düğümüne 2 düğümünü bağlayacağız. Kaynak (`src`) 0, hedef (`dest`) 2'dir. Önce 2 için `createNode` ile yer tutulur, sonra düğüm listeye eklenir.

Hoca burada bir soru sordu. 0'ın listesi 2, 3, 1 iken 4 geldi. 4'ü başa mı eklemek etkilidir, sona mı? Öğrenciler "başa" dedi. Gereksiz yere listenin sonuna kadar ilerleyip vakit kaybetmektense başa eklemek gerekir. Bu, ilk haftalarda bağlı listelerde görülen işlemin aynısıdır. Adımlar şöyledir:

- Yeni düğümün `next` alanı, kaynağın gösterdiği adres olur. Yani 4, 2'yi gösterir.
- Kaynağın liste başı artık yeni düğümü gösterir. Yani 0, 2'yi değil 4'ü gösterir.

```
Önce:   0 -> 2 -> 3 -> 1
Sonra:  0 -> 4 -> 2 -> 3 -> 1
```

Graf yönsüz olduğu için iş burada bitmez. 0'ı 4'e bağladığınız gibi 4'ü de 0'a bağlamanız gerekir. Bu yüzden aynı işlem öbür taraf için de yapılır. `addEdge` içindeki ilk üç satır kaynağı hedefe, son üç satır hedefi kaynağa bağlar.

14.5 Tam Program

Aşağıdaki `struct` tanımları, `createNode`, `createGraph` ve `addEdge` slayttaki koddur. Slaytta olmayan `#include` satırları, `printList`, `freeGraph` ve `main` programın çalışabilmesi için eklendi (ara adım eklendi). `main` içindeki kenarlar hocanın 8.2 bölümündeki grafı ve yukarıdaki başa ekleme örneğidir. Program `gcc -Wall` ile uyarısız derlendi ve çalıştırıldı.

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int vertex;
    struct node* next;
};

struct Graph
{
    int numVertices;
    struct node** adjLists;
};

struct node* createNode(int v)
{
    struct node* newNode = malloc(sizeof(struct node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

struct Graph* createGraph(int vertices)
{
    int i;
    struct Graph* graph = malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    graph->adjLists = malloc(vertices * sizeof(struct node*));
    for (i = 0; i < vertices; i++)
        graph->adjLists[i] = NULL;
    return graph;
}

void addEdge(struct Graph* graph, int src, int dest)
{
```

```c
    struct node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}

/* Bundan sonrası slaytta yoktur: programı çalıştırmak için eklendi. */
void printList(struct Graph* graph, int v)
{
    struct node* p;
    printf("%d:", v);
    for (p = graph->adjLists[v]; p != NULL; p = p->next)
        printf(" %d", p->vertex);
    printf("\n");
}

void freeGraph(struct Graph* graph)
{
    int i;
    for (i = 0; i < graph->numVertices; i++)
    {
        struct node* p = graph->adjLists[i];
        while (p != NULL)
        {
            struct node* q = p->next;
            free(p);
            p = q;
        }
    }
    free(graph->adjLists);
    free(graph);
}

int main(void)
{
    int i;
    struct Graph* g = createGraph(5);

    /* Hocanın beş düğümlü grafı. Başa ekleme yüzünden sıra tersten verildi. */
    addEdge(g, 2, 4);
    addEdge(g, 1, 4);
    addEdge(g, 0, 3);
    addEdge(g, 0, 2);
    addEdge(g, 0, 1);
    for (i = 0; i < 5; i++)
        printList(g, i);
    freeGraph(g);

    /* Hocanın başa ekleme örneği: 0'ın listesi 2, 3, 1 iken 4 gelir. */
    g = createGraph(5);
    addEdge(g, 0, 1);
    addEdge(g, 0, 3);
    addEdge(g, 0, 2);
    printList(g, 0);
    addEdge(g, 0, 4);
    printList(g, 0);
    printList(g, 4);
    freeGraph(g);
```

```
    return 0;
}
```

Çıktı:

```
0: 1 2 3
1: 0 4
2: 0 4
3: 0
4: 1 2
0: 2 3 1
0: 4 2 3 1
4: 0
```

> **Ders sonrası not:** Slayttaki kodda `malloc` sonucu kontrol edilmiyor. `malloc` için `<stdlib.h>` gerekir. Gerçek kodda `NULL` kontrolü yapılmalıdır.

14.6 Ağırlık Nereye Yazılır?

Hoca kodun üzerinden yeniden geçerken düğüme `int weight` alanı ekledi: `vertex` numarayı, `weight` bağlantı değerini, `next` sonraki adresi gösterecekti. Sonra bunu geri aldı. Gerekçesi, 3 numaralı düğümün hem 0'a hem de 1'e bağlı olacağı, bu yüzden ağırlığın düğüme yazılmaması, kenar eklenirken ele alınması gerektiğiydi. Slayttaki basit sürümde yapı `vertex` ve `next` olarak kaldı.

> **Ders sonrası not:** Hoca ağırlığın liste elemanına yazılmasını hata olarak geri aldı; ancak bu kodda ağırlık o elemana yazılabilir. `addEdge` her kenar için `createNode` ile yeni bir eleman oluşturur. 0'ın listesindeki 3 elemanı ile 1'in listesindeki 3 elemanı ayrı kayıtlardır, dolayısıyla ayrı ağırlık taşıyabilirler. Bu, hocanın 8.3 bölümünde anlattığı üç parçalı yapının kendisidir. Karışıklık, aynı elemanın iki listede paylaşıldığı varsayılırsa doğar. `struct` ve `createNode` için derlenip denenen hâl şöyledir:

```
struct node
{
    int vertex;
    int weight;
    struct node* next;
};

struct node* createNode(int v, int w)
{
    struct node* newNode = malloc(sizeof(struct node));
    newNode->vertex = v;
    newNode->weight = w;
    newNode->next = NULL;
    return newNode;
}
```

15. Kruskal İçin Etiket Dizisi Tasarımı

Hoca Kruskal'a geri dönüp tasarımı durum durum çıkardı. Hedef, en küçük kapsayan ağacı oluşturacak kenarları alırken döngü oluşturacakları almamaktır. Bir u düğümü ve bir v düğümü için, hangi alt grafa bağlı olduklarını gösteren bir `label` dizisi tutulur. Bir de etiket sayacı vardır (hocanın yazımıyla `L`), başlangıçta 0'dır.

15.1 Dört Durum

- **Durum 1, `label[u] == 0` ve `label[v] == 0`:** İkisi de hiçbir alt grafa dahil değildir. Sayaç bir artırılır ve ikisi bu yeni etiketle işaretlenir. Çünkü yeni bir alt graf oluşturmuşlardır. Örnek: A-D, sonra C-E.
- **Durum 2, etiketlerden yalnız biri 0:** Sıfır olan düğüm boşluktadır. O düğüm, diğerinin etiketiyle etiketlenir, yani diğerinin alt grafına eklenir. Örnek: D-F, A-B, E-G.

- **Durum 3, `label[u] != label[v]` ve ikisi de sıfırdan farklı:** İki düğüm farklı alt graflardadır. Birinin etiketini taşıyan tüm düğümler diğerinin etiketiyle etiketlenir. B-E'yi bağlıyorsanız E'ye bağlı olan C'yi de bu alt grafa bağlıyorsunuz demektir.
- **Durum 4, `label[u] == label[v]` (sıfırdan farklı):** İşlem yapılmaz. İki düğüm zaten aynı alt graftadır, bir de bunları bağlarsanız mutlaka döngü oluşur. Örnek: B-C, E-F, B-D.

Hoca son durumu öğrencilere sordu.

Soru: Son durum ne olabilir?

Cevap: Bir öğrenci, etiketlerin sıfırdan farklı ve döngü oluşturma durumu olacağını söyledi. Hoca bunu `label[u] == label[v]` diye netleştirdi: Döngü oluşup oluşmadığı bu eşitlikten anlaşılır.

15.2 Ne Zaman Durulur?

Soru: `while` döngüsü içinde "kenar sayısı, düğüm sayısından bir eksik olana kadar yap" dersek olur mu?

Cevap: Aynen öyle. n düğüm varsa $n-1$ kenar alınınca durulur. Bu örnekte 7 düğüm için 6. kenar olan E-G alındığında daha fazla ilerlenmez.

Tasarım şöyle özetlenebilir. Sözde kod, hocanın dört durumundan ve öğrencinin döngü önerisinden kuruldu (ara adım eklendi). Python ile derste çözülen örnekte çalıştırıldı ve yukarıdaki tabloyu aynen verdi. Ayrıca 3000 rastgele bağlı grafta Prim ile aynı toplamı verdi.

```
label[] dizisinin tamamı = 0
L = 0, alinan = 0, i = 0
kenarlar ağırlığa göre küçükten büyüğe sıralı
while alinan < n - 1:
    (u, v) = i. kenar
    if label[u] == 0 ve label[v] == 0:
        L = L + 1; label[u] = L; label[v] = L; alinan = alinan + 1
    else if label[u] == 0:
        label[u] = label[v]; alinan = alinan + 1
    else if label[v] == 0:
        label[v] = label[u]; alinan = alinan + 1
    else if label[u] != label[v]:
        eski = label[v]
        etiketi eski olan tüm düğümleri label[u] yap; alinan = alinan + 1
    (label[u] == label[v] ise kenar alınmaz)
    i = i + 1
```

> **Ek bilgi (derste anlatılmadı):** Etiket dizisi, ayrık küme (disjoint set, union-find) yapısının basit bir hâlidir. İki düğümün aynı alt grafta olup olmadığına bakmak $O(1)$ sürer. Durum 3'te bir etiketi taşıyan düğümleri bulmak için dizi taranır, bu da $O(V)$ sürer ve en fazla $V-1$ kez olur. Sıralama ile birlikte toplam maliyet $O(E \log E + V^2)$ olur.

15.3 Kendi Başınıza Kurun

Hoca öğrencilerden bu tasarımı kendilerinin kurmasını istedi. Önce girişin sıralı olduğunu varsayın, grafın bağlantılarını dışarıdan sıralı okuyun. Hocaya göre sıralamayla uğraşmanın anlamı yoktur, hazır bir sıralama fonksiyonu çağırmak yeterlidir. Kenarları da vermek gerekir: "A ile D'yi bağladım, bağlantının değeri 5" gibi.

> **Hocanın vurgusu:** Artık pek çok algoritmayı rahatça yazabilecek durumdasınız. Asıl zorluk, algoritmayı veri yapılarıyla birleştirmeye çalıştığınızda çıkıyor. Bu tasarımı kendi başınıza kurarsanız konuyu yerine oturtursunuz. `createNode`, `addEdge` gibi fonksiyonları kendiniz yazmazsanız hiçbir kalıcılığı olmaz.

16. Yönlü Grafta Farklı Ağırlıklar ve Kendine Dönen Kenar

Soru: Yönlü bir grafta hem A'dan D'ye hem D'den A'ya kenar olsun. Bir yönde ağırlık 5, diğer yönde 7 olan bir örnek var mı?

Cevap: Evet, olabilir. Örneğin Beşiktaş'tan Şişli'ye giderken yol 3 km tutuyor olabilir. Yol çift yönlü olmadığı için dönüşte başka bir yol kullanılır ve o yol 4 km tutabilir. Gidiş ve dönüş uzunlukları farklı olabilir. Yönlü bir graf üzerinde çalışıyorsanız buna dikkat etmek zorundasınız.

Ayrıca bir düğüm kendine de dönebilir, yani kendine dönen bir kenar (self loop) olabilir. Bu derste ayrıca işlenmedi.

Dersin Toparlaması

Bu derste graf terminolojisi hatırlatıldı: düğüm, kenar, derece, yol, döngü, bağlı bileşen, ağaç, iki parçalı graf, ağırlıklı, yönsüz ve yönlü graf. Ardından grafın bilgisayarda nasıl tutulacağı işlendi. Adları sayılara çeviren sembol tablosu, V^2 yer isteyen ama bağlantı sorgusunu $O(1)$'de yanıtlayan komşuluk matrisi ve seyrek graflar için uygun olan komşuluk listesi karşılaştırıldı. Seçimin tasarlanan algoritmanın ihtiyacına göre yapılacağı vurgulandı. Komşuluk listesinin `createNode`, `createGraph`, `addEdge` fonksiyonlarıyla C kodu görüldü.

İkinci ana konu en küçük kapsayan ağaçtı. Prim, bir düğümden başlayıp ağacı büyüttü. Kruskal, sıralı kenarları etiket dizisi ve dört durumla işledi. Aynı 7 düğümlü grafta ikisi de 6 kenarlı ve toplamı 39 olan aynı ağacı verdi. Hoca Kruskal'ın algoritmik tasarımının kendi başına kurulmasını istedi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.